

C2ISOLATE

Erkennung von C2 Verhalten im Netzwerk mit
ML-Algorithmen

Silas Burkhard

Inhaltsverzeichnis

1 Einleitung.....	3
1.1 Änderungstabelle	3
1.2 Projektbeschreibung.....	3
1.3 Risiken.....	3
2 Hauptteil.....	3
2.1 C2 Kommunikationstechniken.....	3
2.2 ML-Algorithmen.....	4
2.2.1 Isolation Forest	5
2.2.2 Random Forest	5
2.2.3 K-means Clustering.....	5
2.2.4 MLP.....	6
2.3 Featureliste.....	7
2.4 Prototyp	12
2.4.1 ML-Kombination	14
2.4.2 Analyse und Überarbeitung.....	17
2.4.3 Rückverfolgung auf Pakete	20
2.5 Testen mit echten C2 Frameworks	21
2.5.1 Sliver Setup	21
Halluzinationen	23
Glossar.....	23
Tagesreflexionen	23
Tag 1, 03.06	23
Tag 2, 04.06	23
Tag 3, 05.06	23
Tag 4, 10.06	23
Tag 5, 11.06	24
Tag 6, 12.06	24
Tag 7, 17.06	24
Tag 8, 18.06	24
Tag 9, 19.06	24
Tag 10, 24.06	24

Tag 11, 25.06	24
Tag 12, 26.06	24

1 Einleitung

1.1 Änderungstabelle

Datum	ÄNDERUNGSBESCHREIB	ORT
03.06.2026	Dokument erstellt	
04.06.2026	Grundlegende Struktur	Einleitung, Hauptteil, Tagesreflexionen
05.06.2026	Kommunikationstechniken und Featureliste	Hauptteil

1.2 Projektbeschreibung

In meinem Projekt will ich C2 Verhalten im Netzwerkverkehr mithilfe von ML-Algorithmen erkennen. Dazu muss ich herausfinden, was für Techniken angreifer nutzen, um ihren Datenverkehr zu verschleiern, was ich für einen ML-Algorithmus nutzen werde und welche Features des Datenverkehrs ich brauche.

Ich kam auf dieses Thema, da ich mir zuerst überlegt hatte, allgemein den Netzwerkverkehr mit Machine Learning zu überwachen, aber dann einfach Port-Scans oder Syn-Floods zu erkennen fand ich langweilig. Deshalb habe ich mich dafür entschieden etwas Konkreteres zu machen, und C2 Verhalten zu erkennen wäre sicher eine Challenge, da dieser Verkehr dazu gedacht ist unerkannt zu bleiben.

1.3 Risiken

Es gibt einige Risiken in meinem Projekt. Zum Beispiel kann es sein, dass ich gar nicht einen geeigneten Algorithmus finde, der mir weiterhilft. Oder es könnte sein, dass der Verkehr den ich als C2 simuliere gar nicht wirklich mit echtem C2 Verkehr vergleichbar ist. Ich weiss auch nicht, welche Features ich brauche, um zuverlässige Resultate zu erzielen. Auch Probleme beim Coden können viel Zeit kosten.

2 Hauptteil

2.1 C2 Kommunikationstechniken

Um später verdächtigen Datenverkehr zu erkennen, muss ich wissen wie eine C2-Struktur kommuniziert. Eine einfache C2-Architektur sieht meistens ungefähr so aus:

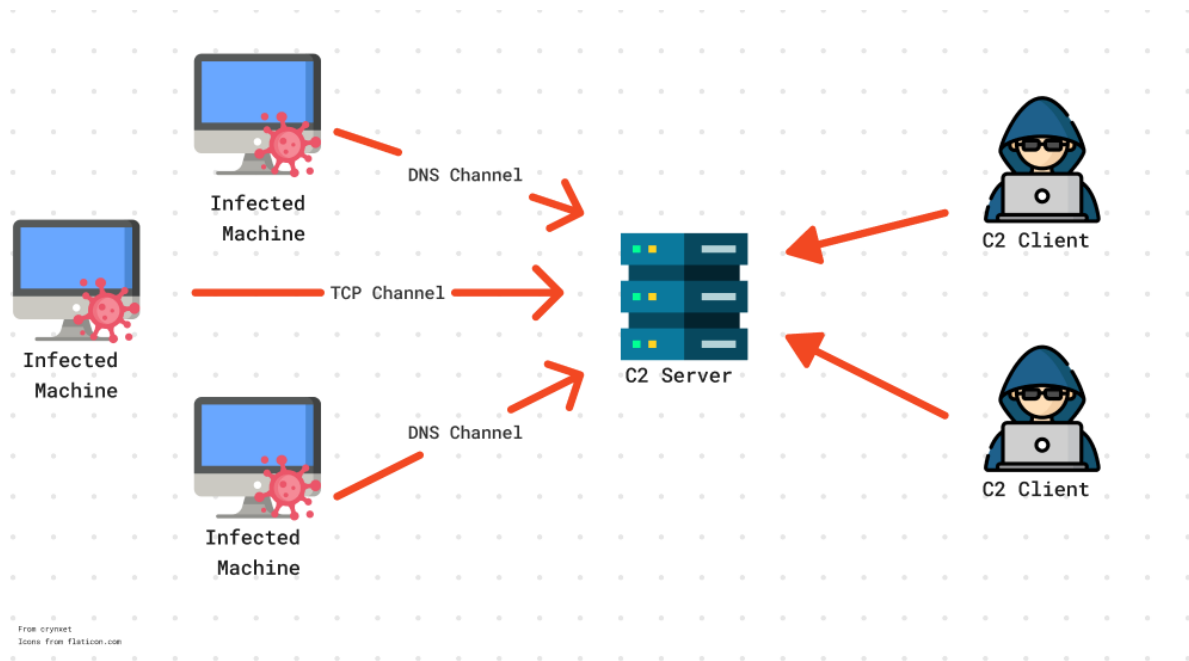


Abbildung 1: C2 Struktur [Quelle](#)

Dabei stehen die infizierten Maschinen direkt im Kontakt mit dem C2 Server. Angreifer können dann über den C2 Server Befehle ausführen oder Angriffe koordinieren. Für die Kommunikation zwischen der infizierten Maschine und dem C2 Server gibt es viele Möglichkeiten. Einige davon sind:

- DNS-Tunneling
- Socket Filter
- Protocol/Service Impersonation

Mit diesen Techniken kann der Datenfluss zwischen dem C2 Server und dem Opfer System versteckt werden. Die meist genutzten Protokolle zur C2 Kommunikation sind HTTP/HTTPS, DNS, IRC, FTP und E-Mail Protokolle.

Eine verbreitete Methode um den Opfersystemen Anweisungen zu schicken, ist Beaconsing. Dabei schickt das Opfer regelmässige Anfragen an den C2 Server um Anweisungen zu bekommen. Das primäre Ziel von diesem Projekt ist es, genau diesen Datenverkehr zu erkennen. Angreifer nutzen Taktiken wie Jitter, wo ein zufälliger Wert der normalen Wartezeit hinzugefügt wird, damit der Datenverkehr weniger Roboterhaft wirkt.

2.2 ML-Algorithmen

Ich habe mich über einige ML-Algorithmen informiert, die sich für mein Projekt eignen könnten. Es gibt noch viel mehr und auch viel spezialisiertere Algorithmen als diese, aber ich habe mich über diese informiert, weil ich schon darüber gehört oder schon damit gearbeitet habe.

2.2.1 Isolation Forest

Isolation Forest ist ein Algorithmus zur Anomalie-Erkennung. Um Anomalien zu finden, wählt der Algorithmus ein Feature von dem Datensatz ausgewählt und dann an einer zufälligen Stelle halbiert. Dieser Prozess der Halbierung wird wiederholt, bis eine von zwei Bedingungen erfüllt sind: Entweder ist an jedem Punkt nur noch ein Datentupel oder es wurde eine vorbestimmte Tiefe erreicht. Jetzt hat man einen Baum von Daten, und an jedem Ende eines Astes hat es nur noch wenige bis ein Datentupel. Anomalien brauchen weniger Spaltungen als die anderen Daten um sich vollständig abzugrenzen, dadurch findet man sie. Ein solcher Baum wird auch wiederholt erstellt, um die Sicherheit des Resultates zu verbessern.

Dieser Algorithmus ist auch gut mit grossen Datenmengen und wird auch in der Cybersecurity verwendet. Ausserdem braucht der Algorithmus kein Training, es ist ein „unsupervised Learning Algorithm“.

2.2.2 Random Forest

Random Forest ist ein Algorithmus, der sich wie Isolation Forest auch Decision Trees zunutze macht. Dabei ist Random Forest nicht wie Isolation Forest auf Anomalien ausgelegt, sondern auf Klassifizierung und Regression. Random Forest ist auch ein supervised Learning Algorithm, es braucht also gekennzeichnete Trainingsdaten.

2.2.3 K-means Clustering

K-means Clustering ist ein Algorithmus zur Gruppierung von Daten. Dabei wird angenommen, dass die Gruppen näherungsweise Spherenförmig sind und man muss die Anzahl von Gruppen wissen. K-means Clustering ist wie der Isolation Forest ein unsupervised Learning Algorithm, und braucht kein Training mit gekennzeichneten Übungsdaten.

Bei K-means Clustering werden zunächst zufällige Startpunkte als Zentrum für die Gruppen ausgewählt. Danach versucht der Algorithmus die Zentren so zu verbessern, dass mehr Punkte einen kleineren Abstand zu ihrem Zentrum haben. Dies wird mehrmals wiederholt, bis man ein stabiles Resultat hat.

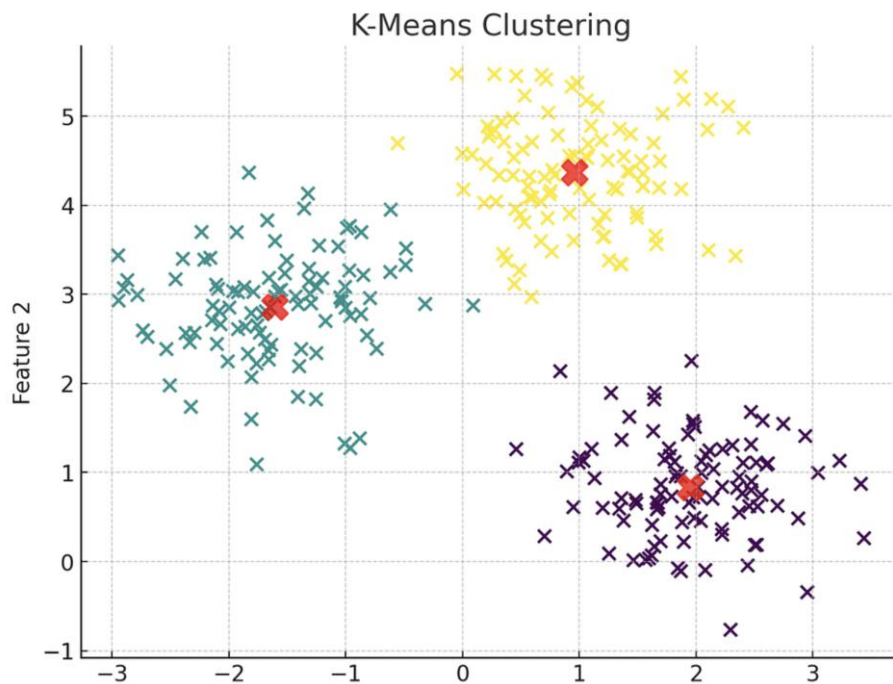


Abbildung 2: K-means Clustering [Quelle](#)

Zuerst dachte ich, dass ich diesen Algorithmus wahrscheinlich noch gut nutzen könnte, jedoch sind meine Daten fast nie zuverlässig Spherenförmig aufgeteilt. Deshalb werde ich diesen Algorithmus eher nicht nutzen.

2.2.4 MLP

MLP steht für Multilayer Perceptron, und ist ein künstliches Neuronales Netzwerk, dem durch Lernen Zusammenhänge zwischen Daten, klassifizierung, Anomalieerkennung und weiteres beigebracht werden kann. Durch das ist es ein sehr ein starkes Tool, wenn man die Trainingsdaten und Rechenleistung dafür hat.

Ein MLP hat immer den gleichen Aufbau: Zuerst eine Schicht von Input-Neuronen, die Anzahl von diesen korrespondiert direkt mit der Anzahl an Dimensionen der Daten, die der MLP entgegennehmen kann. Dann kommt eine definierte Anzahl an verborgenen Schichten, bei diesen werden schlussendlich die Kalkulationen gemacht. Am Ende ist noch eine letzte Schicht von Neuronen, der Output-Layer. Die Struktur kann zum Beispiel wie im Bild unten aussehen:

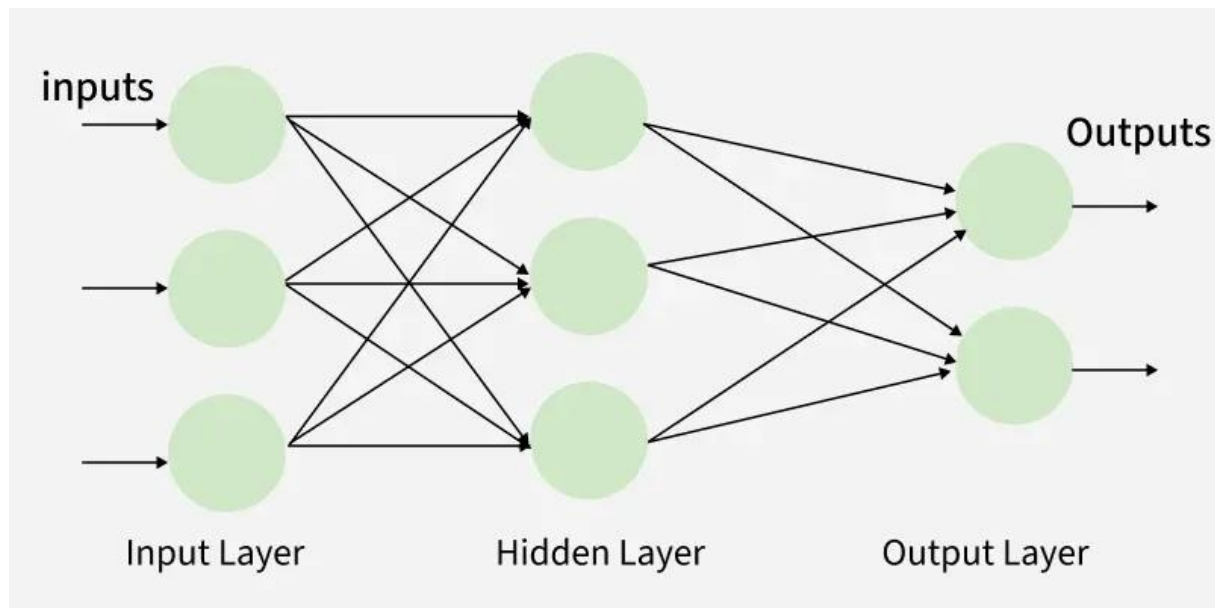


Abbildung 3: Struktur eines MLP [Quelle](#)

Ich habe mich eher gegen den Multilayer Perceptron entschieden, da ich dann grosse Mengen von Übungsdaten haben müsste, um ein Model zu trainieren. Es ist aber sicher eine interessante Möglichkeit das Problem anzugehen, wenn man genügend Zeit und Informationen hat.

2.3 Featureliste

Da ich mich für die Python Library `nfstream` entschieden habe, bekomme ich nun pro Flow 86 Features über diesen Flow. Vieles davon wird nicht aufzeigen, ob ein Flow eine Kommunikation mit einem C2 Server ist. Deshalb muss ich ein Verfahren haben, um für jedes Feature zu prüfen, ob es mir hilft, bösartiger Verkehr zu erkennen.

Aus diesem Grund habe ich mich dafür entschieden eine Dimensionsreduzierende Funktion zu nutzen, wodurch man die Daten auf einen Zwei-Dimensionalen Graph visualisieren kann. Zuerst wollte ich `t-SNE` nutzen, da ich über dieses Verfahren schon in meiner Recherche über die verschiedenen ML-Algorithmen gestossen bin. Aber nachdem ich nochmals ein wenig darüber nachgeforscht habe, habe ich herausgefunden, dass `t-SNE` nicht sehr gut ist für sehr grosse Datensets. Deshalb bin ich auf `PCA`, Principal Component Analysis, umgestiegen. `PCA` ist linear, nicht so wie `t-SNE`. Ich implementierte `PCA` in meinem Script, sodass ich am Ende einen Graphen zeichnen konnte, auf dem alle Flows sichtbar sind. Die Flows die von meinem C2 Traffic Simulator waren, habe ich rot gezeichnet, den Rest grau. Nun kann ich Schritt für Schritt Features entfernen und den Graph vergleichen. Wenn sich die roten Punkte besser gruppieren und sich von den grauen Punkten abgrenzen, ist das Feature hinderlich zum C2 Traffic zu erkennen, und ich werde es permanent entfernen. Wenn sich am Graph nicht wirklich etwas verändert, ist das Feature auch irrelevant und wird entfernt. Wenn aber sich die Gruppierung der roten Punkte verschlechtert und sie besser einblenden ist das Feature relevant, und ich werde es nicht entfernen.

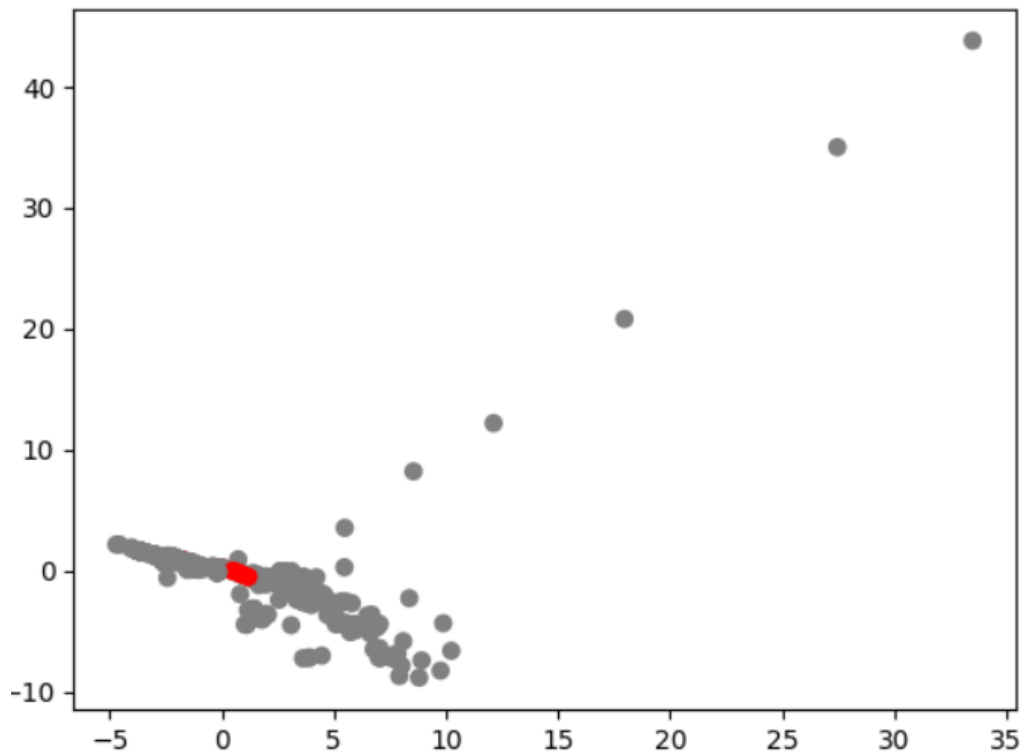


Abbildung 4: Ursprungsgraph mit allen Features

Dieser erste Graph wurde mit allen Features erstellt. Man sieht, dass sich die roten Punkte gut mit den grauen vermischen, was nicht gut ist.

Ein paar Features streiche ich schon, ohne sie zu überprüfen, da ich weiss, dass sie nicht relevant sind. Folgende Features habe ich gestrichen:

- id
- expiration_id
- ip_version
- vlan_id
- tunnel_id
- application_is_guessed
- application_confidence

Ohne diese Features sieht der Graph so aus:

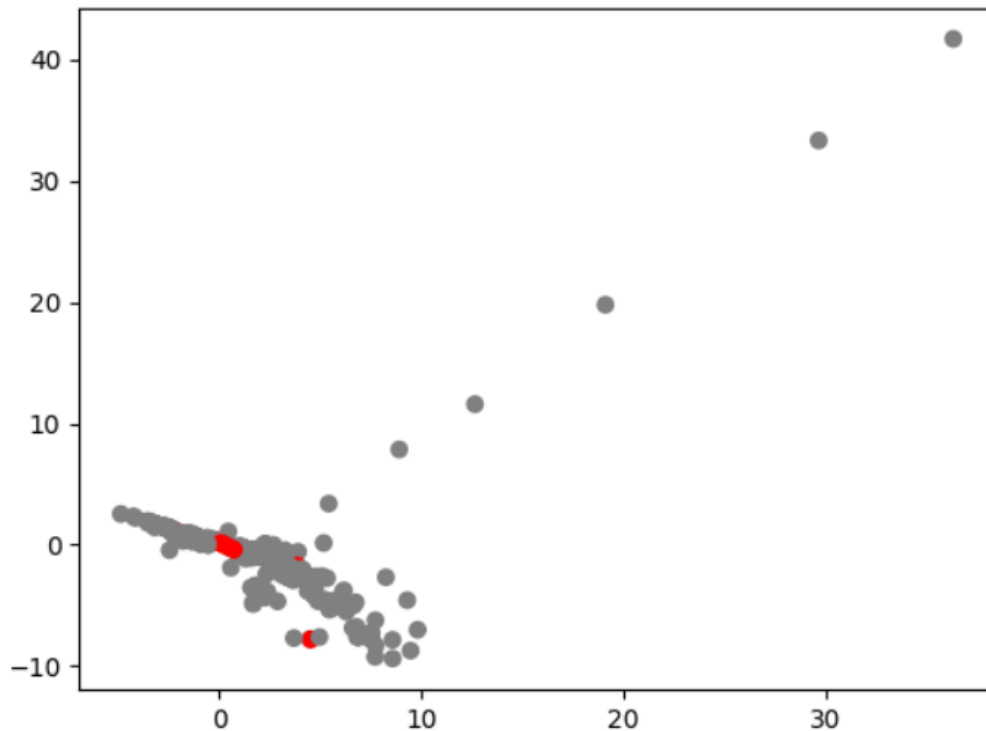


Abbildung 5: Graph ohne unwichtige Features

Im Vergleich zum ersten Graph hat sich nicht sehr viel verändert. Dadurch merkt man wie oben beschrieben, dass diese Features wie ich angenommen habe, unwichtig sind.

Nachdem ich diesen Prozess ein paarmal wiederholt habe, ist mir aufgefallen, dass meine gekennzeichneten roten Punkte gar nicht stimmen können. Ich nutzte ChatGPT um den Fehler in meinem Code zu finden, ich hatte verschiedene Arten von Indexen genutzt. Einerseits der Index von `nfstream`, der nicht unbedingt mit dem positionellen Index von Arrays übereinstimmt. Andererseits auch noch der Index vom Pandas Objekt `DataFrame`, der auch nicht mit den anderen zwei Indexen übereinstimmen muss. Nachdem ich dieses Problem gelöst hatte und weitere Features überprüft habe, fiel mir auf, dass jedes Feature das ich entfernte oder hinzufügte fast keinen Einfluss darauf hatte, wie gut die roten Punkte positioniert waren.

Deshalb entschied ich mich, es noch mit dem t-SNE Algorithmus zu versuchen, da die Linearität von PCA ein Problem sein könnte. Beim Graph unten sieht man, dass die Punkte schon viel besser gruppiert waren.

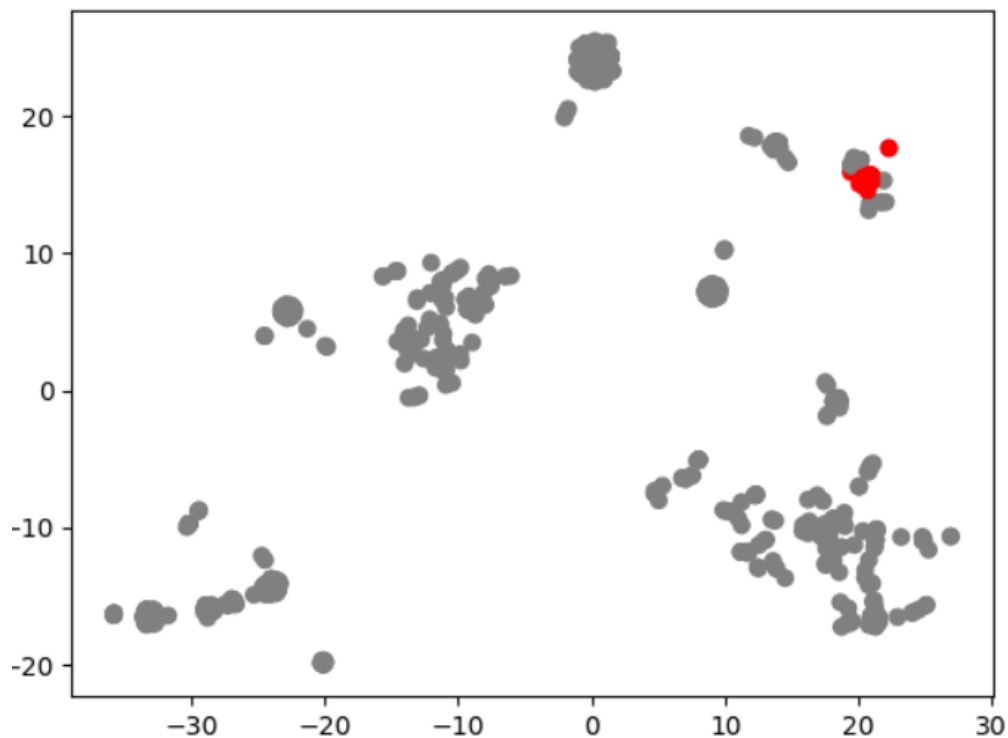


Abbildung 6: t-SNE mit fast allen Features

Nachdem ich viele verschiedene Varianten ausprobiert habe, habe ich mich für die folgenden Features entschieden:

- `bidirectional_duration_ms`: Dauer des Flows von beiden Seiten in Millisekunden
- `bidirectional_packets`: Anzahl an Paketen im Flow
- `bidirectional_bytes`: Anzahl an Bytes im Flow
- `src2dst_bytes`: Anzahl an src2dst Bytes im Flow
- `dst2src_bytes`: Anzahl an dst2src Bytes im Flow
- `dst2src_max_ps`: Maximale dst2src Paketgröße
- `src2dst_max_ps`: Maximale src2dst Paketgröße
- `src2dst_max_piat_ms`: Maximale Zeit zwischen src2dst Paketen
- `dst2src_max_piat_ms`: Maximale Zeit zwischen dst2src Paketen

Ich habe Features gruppenweise auf ihren Einfluss überprüft. Zum Beispiel alle Features die nur die Anzahl einer bestimmten Flag zählen oder alle Features die die Startzeit des Flows beinhalten. Graphen mit den ausgewählten Features sehen so aus:

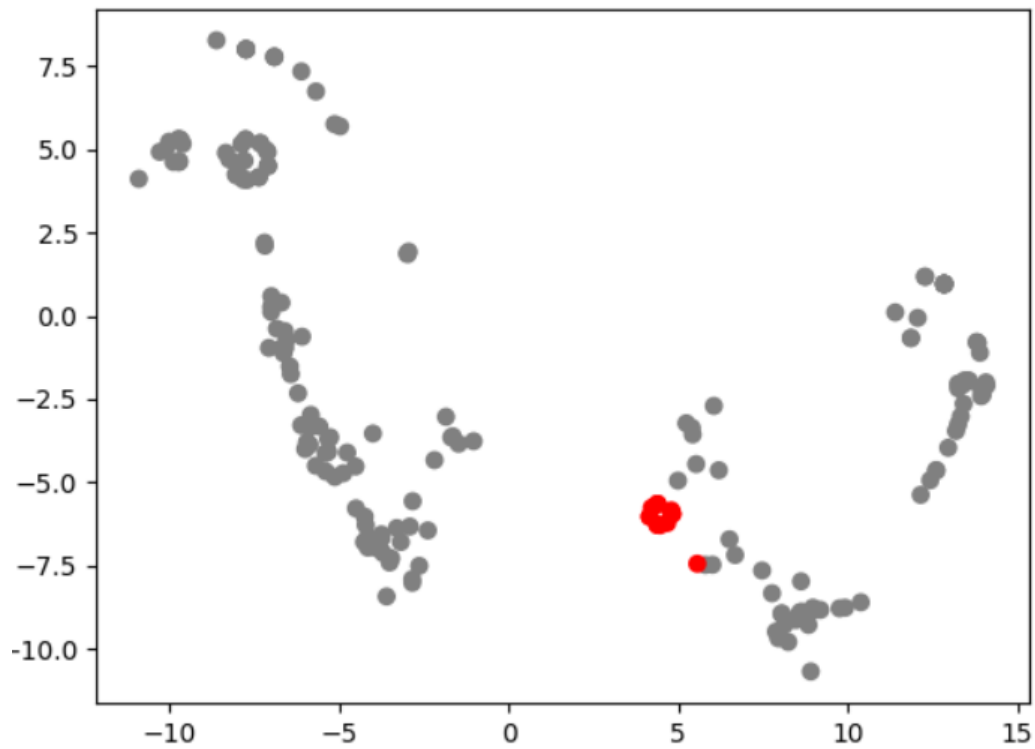


Abbildung 7: Features mit 5k Paketen

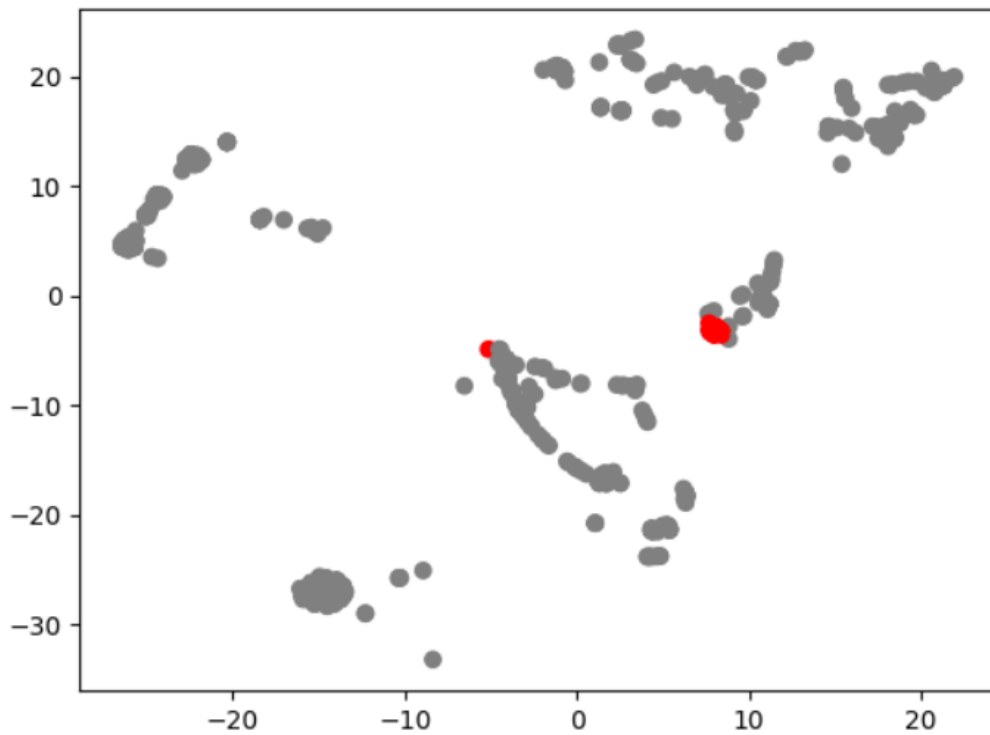


Abbildung 8: Features mit 400k Paketen

Man sieht, dass die roten Punkte sich gruppieren, es ist aber auch keinesfalls perfekt. Ich werde später im Projekt die Liste der ausgewählten Features auch noch verändern.

2.4 Prototyp

Mein zweiter Meilenstein ist, ein Prototyp zu schreiben, der mit Testdaten den böartigen Traffic richtig identifizieren kann. Ich habe meinen Code in mehrere Files aufgeteilt, damit es besser strukturiert ist. Der allgemeine Ablauf meines Prototyps sieht wie folgt aus:

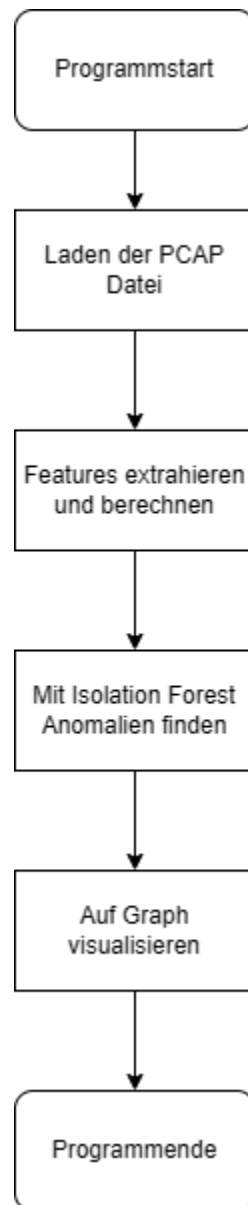


Abbildung 9: Programablauf

Mit diesem Ablauf bekam ich überhaupt keine guten Resultate. Die roten Punkte waren sehr mit den grauen vermischt, und der Isolation Forest zeigte nur normale Pakete als Anomalien an. Selbst mit den gut ausgewählten Features, gab es keine gute Trennung zwischen den roten und den grauen Punkten.

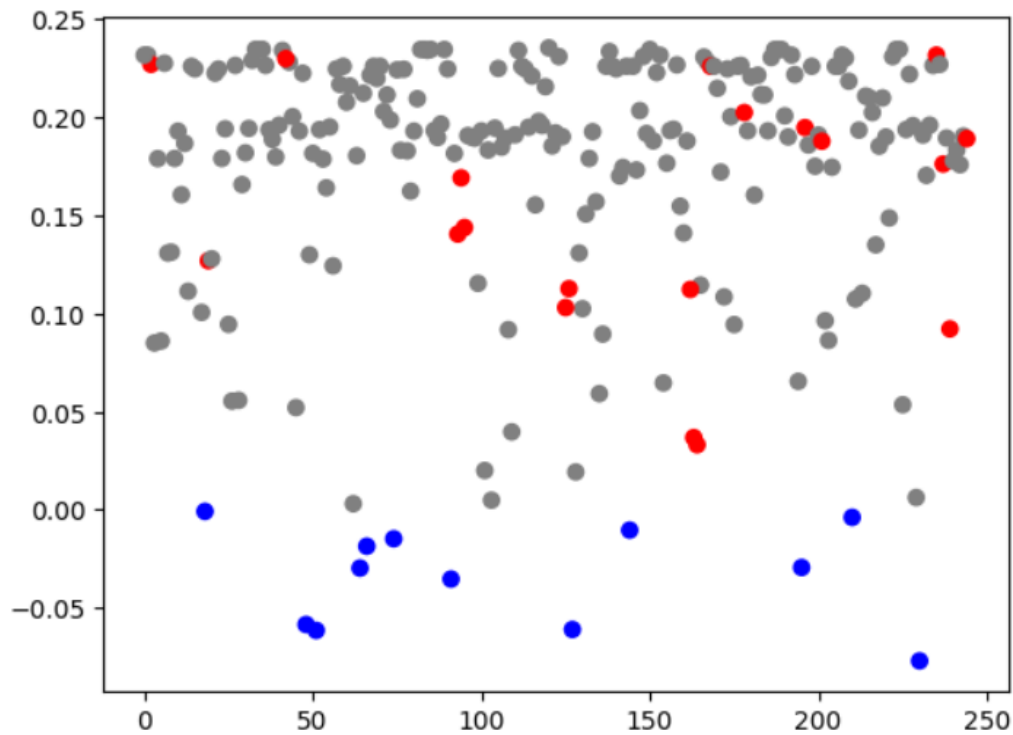


Abbildung 10: Isolation Forest ohne optimierte Features

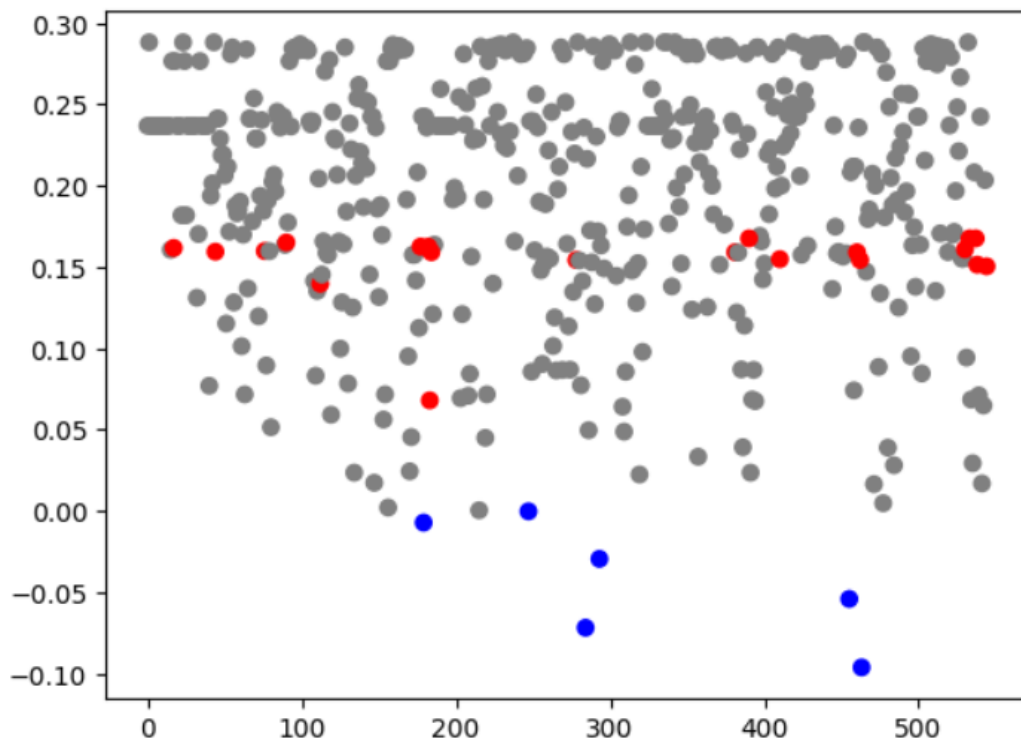


Abbildung 11: Isolation Forest mit optimierten Features

Zwar sieht man klar, dass die roten Punkte weiter unten sind (Die Höhe der Punkte entspricht dem Anomalie-Score, desto tiefer dieser Score, um so wahrscheinlicher ist es eine Anomalie), aber sie sind trotzdem noch komplett umgeben von grauen Punkten.

Zuerst überprüfte ich nochmals, ob die Daten, die angezeigt wurden auch so stimmen, aber dort habe ich keine Probleme gefunden. Danach bin ich nochmals alle Features aus meiner Featureliste durchgegangen, und habe zu jedem wieder ein Graph gezeichnet, um zu überprüfen, ob ich dort wichtige Features übersehen hatte. Ich konnte ein paar Verbesserungen machen, aber auch dies lieferte keine besseren Endresultate. Ich probierte auch den K-means Clustering Algorithmus zu verwenden, sodass ein Cluster die verdächtigen Pakete waren, aber dies war sehr Inkonsistent. Es konnte sein dass in einem versuch perfekt alle roten Punkte in einem Cluster waren, und im nächsten waren alle wieder verteilt.

2.4.1 ML-Kombination

Als ich die neuen Features aber noch zusammen mit t-SNE genutzt habe, um eine Ansicht ohne Isolation Forest zu haben, fiel mir auf, dass hier die roten Punkte sehr gut beieinanderliegen. Auch hatten sie zu den meisten grauen Punkten einen guten Abstand.

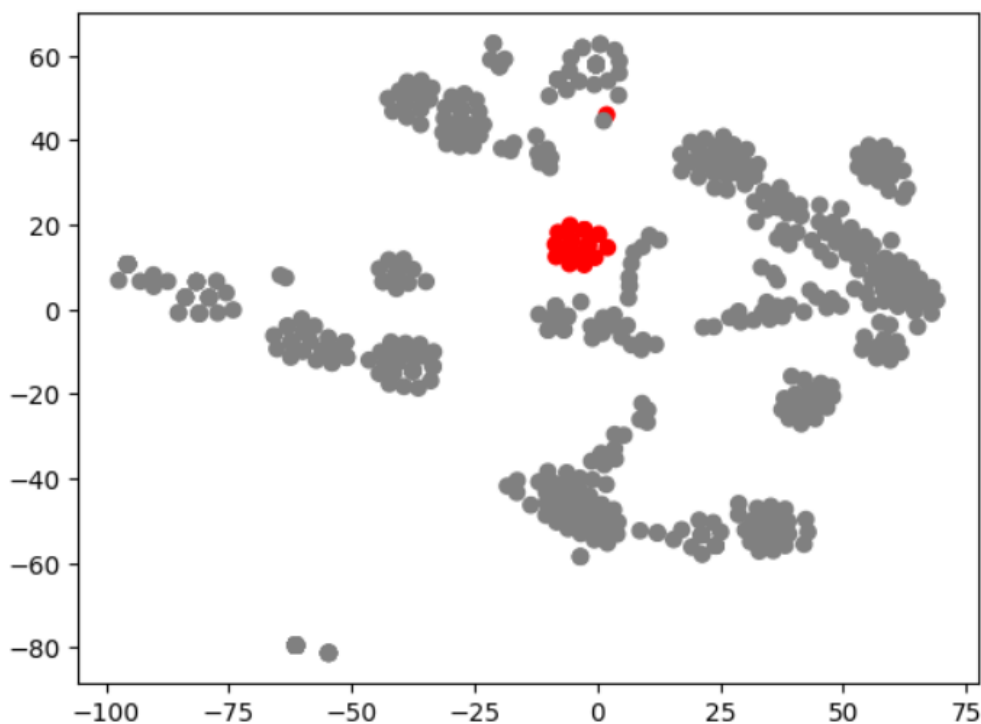


Abbildung 12: Verbesserte Features mit t-SNE

Für dieses Bild wurden folgende Features benutzt:

- time_since_prev_flow
- protocol_id
- dst2src_max_ps
- dst2src_stddev_ps

Ich habe also von meiner eigentlichen Featureliste ein paar Features weggelassen. Ich habe dann ein wenig später die Features dst2src_max_ps und dst2src_stddev_ps wieder

hinzugefügt, da sie die Berechnungen nicht unbedingt verbesserten, aber konsistenter machten.

Wie man in der Grafik oben sieht, sind fast alle Punkte Spheren-förmig. Ich hatte am Anfang dieses Projektes auch einen Algorithmus angeschaut, der sehr gut im Gruppieren von Spherenförmigen Daten ist: K-means Clustering. Also probierte ich, K-means auf die Ausgabe von t-SNE zu verwenden, um diese Datenpunkte auch logisch zu unterteilen. Als ich dann die Ausgabe von K-means wieder grafisch darstellte, konnte ich fast alle Punkte demselben Cluster zuweisen. Dieses Ergebnis war auch sehr konsistent, nicht wie als ich probiert habe, K-means direkt anzuwenden.

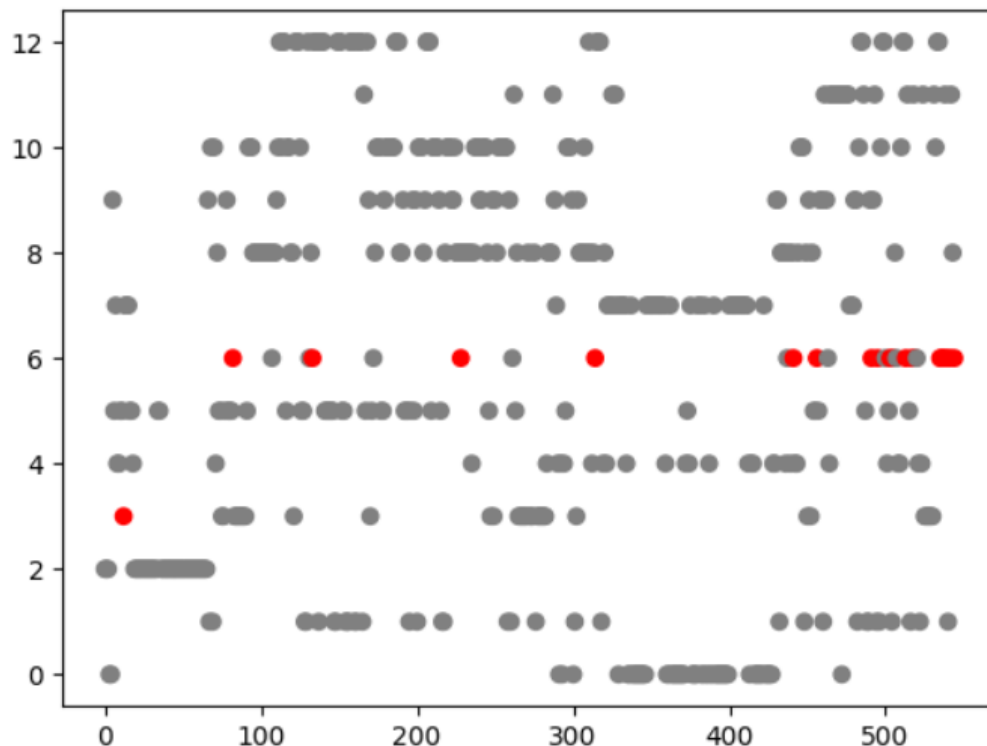


Abbildung 13: t-SNE mit K-means Clustering kombiniert

Bei diesem Graph korrespondiert die Höhe mit der Nummer des Clusters. Alle roten Punkte ausser einem sind in einem einzelnen Cluster, welches in sich selbst nicht viele graue Punkte hat.

Leider kann man von dieser Grafik die roten Punkte nicht erkennen, wenn man sie nicht im vorhinein markiert hat. Darum habe ich auch noch Isolation Forest angewendet, damit man weiss, welches Cluster das böartige ist. Damit das funktioniert, habe ich einen neuen leeren Datensatz erstellt und für jedes Cluster für jedes Feature die Standardabweichung und den Durchschnitt berechnet. Diesen Datensatz, der nur noch Cluster enthält, gab ich als Eingabe für Isolation Forest. Die Resultate waren zum ersten mal nicht schlecht:

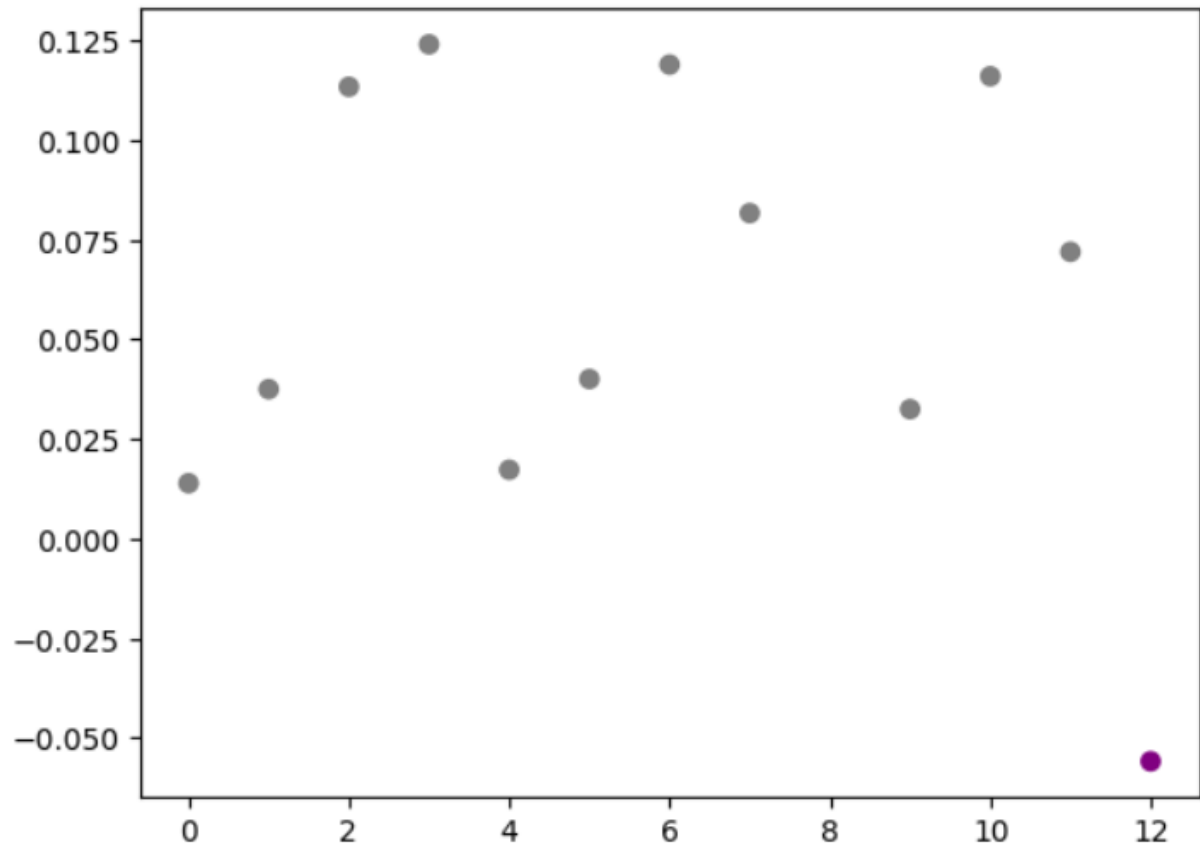


Abbildung 14: Erfolgreiche Erkennung von C2 Verhalten

Mein Prototyp hat das richtige Cluster als bösartig erkannt und markiert. Der Programmablauf des Prototyps, hat sich durch all diese Erkenntnisse auch verschoben, das Diagramm dazu sieht so aus:

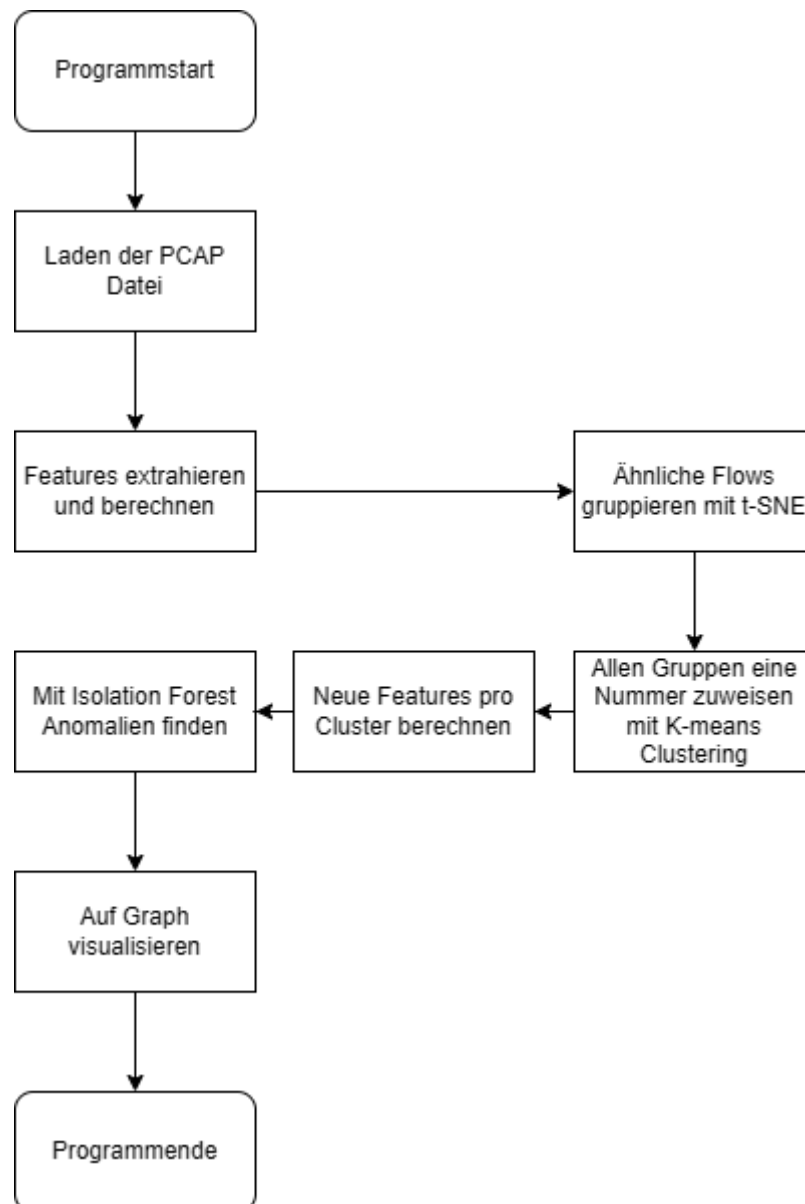


Abbildung 15: Neuer Programmablauf

Ich habe mit diesem Prototyp verschiedene Capture Files ausprobiert, und mich darauf geachtet, ob es überall den C2-Verkehr erkennen kann. Bei einer spezifischen sehr grossen Datei hatte ich lange Probleme, dass der Prototyp den C2-Verkehr nicht als Anomalie erkannt hat. Ich habe viel Zeit darin investiert, das Problem zu beheben, aber am Ende fand ich im Capture File heraus, dass ein Windows Service spezielle Datenverbindungen gemacht hat, die aussergewöhnlicher waren als die C2 Verbindungen.

2.4.2 Analyse und Überarbeitung

Als nächstes schrieb ich ein Auswertungsprogramm, welches den Prototyp mehrfach laufenlässt, und die Ergebnisse anzeigt. So kann man den Prototyp testen und auch Anpassungen bei den ML-Algorithmen werden sichtbar.

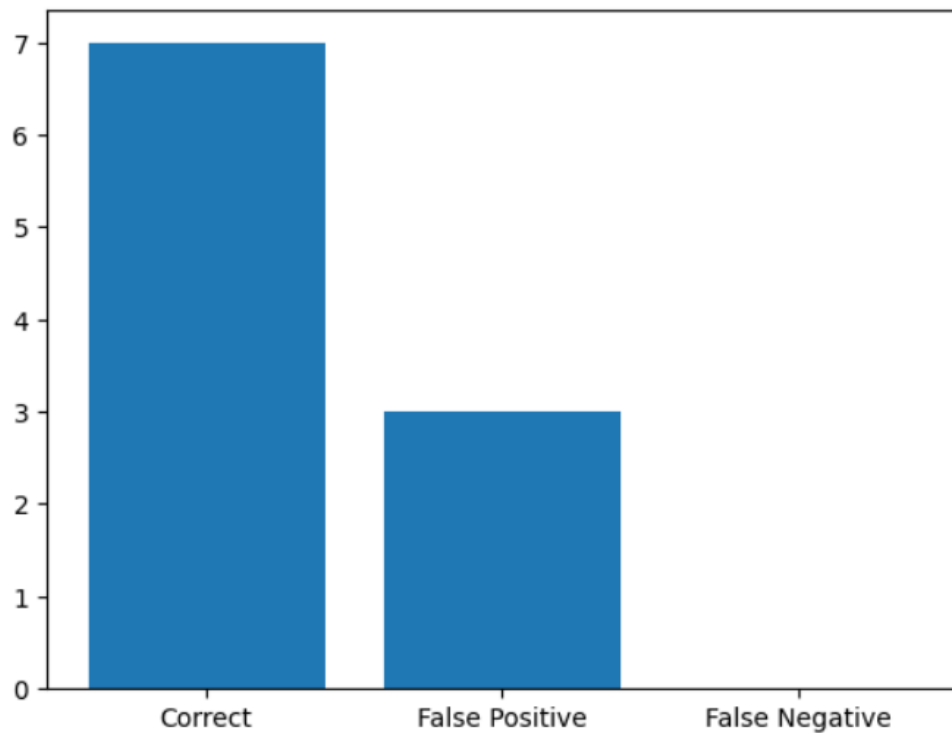


Abbildung 16: Auswertung mit 10 Iterationen

Dabei ist es wichtig, dass mehr iterationen genauere Resultate liefern werden. Zum Beispiel gibt das Diagramm unten viel sichere Angaben, da es 100 Iterationen nutzt und nicht nur Zehn.

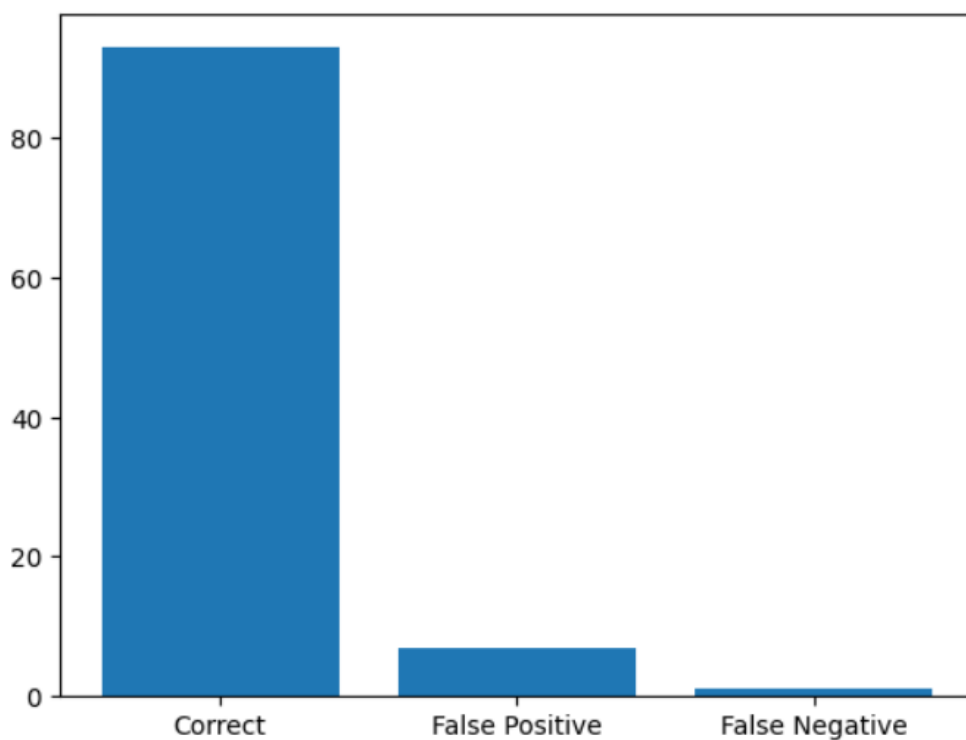


Abbildung 17: Auswertung mit 100 Iterationen

Der Prototyp hat bei diesem Capture File 93 mal richtig den C2 Verkehr erkannt, hat 7 mal normalen Verkehr als böartig bezeichnet und 1 mal hat es ein böartiges Cluster nicht gefunden.

Leider funktionierte die Erkennung nicht mehr gut, sobald ich ein Capture File nutzte, wo ich Jitter hinzugefügt hatte, um es realistischer zu machen, erkannte der Prototyp nichts mehr:

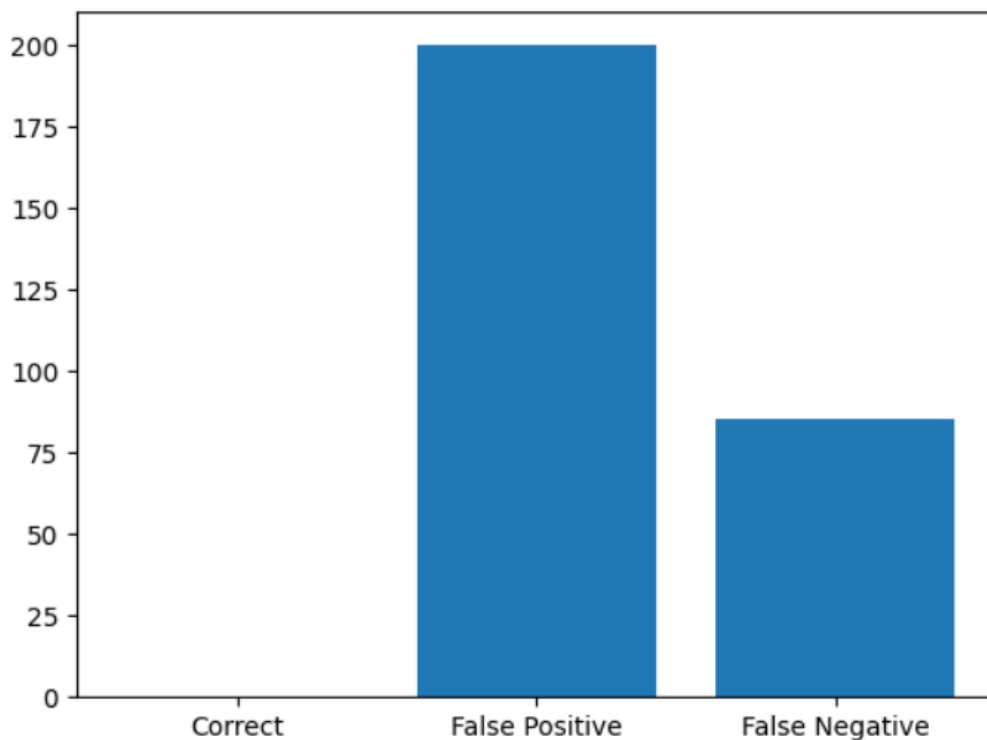


Abbildung 18: Prototyp mit Jitter

Das bedeutete, dass meinem Prototyp noch weitere Features fehlten, die unabhängig von der Zeit zwischen ähnlichen Flows sind. Da ich bei den Cluster gemerkt habe, dass die roten Punkte viel dichter zusammen sind als die restlichen Cluster, hatte ich die Idee, die Cluster-dichte als Feature zu nehmen. Dazu musste ich eine Funktion schreiben, die die durchschnittliche Dichte pro Cluster berechnet. Mithilfe diesem Feature wurde die Erkennung um einiges verbessert, da es vor allem auf dem t-SNE und K-means Algorithmus basiert, und nicht nur direkt auf Isolation Forest. Das Diagramm mit den Verbesserungen sieht so aus:

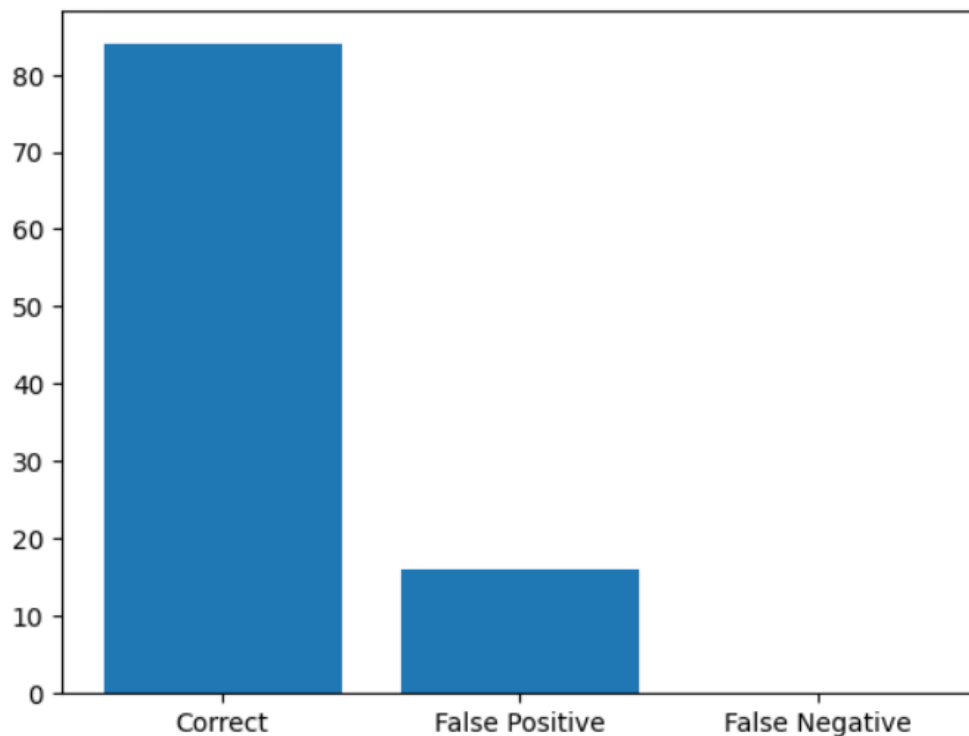


Abbildung 19: Erkennung mit Dichte

Da die Dichte auch auf den anderen Features basiert, konnte es den Simulierten Verkehr viel besser erkennen. Mit dem neuen Programmablauf und Dichte als Feature erkannte mein Programm bei allen Simulierten Aufnahmen einigermaßen Zuverlässig.

2.4.3 Rückverfolgung auf Pakete

Mein Prototyp konnte nun aus einer Capture Datei C2 Verhalten erkennen und graphisch darstellen. Nur hat man von dieser Ansicht keinen Weg nachzuverfolgen, welche Pakete als Anomalien erkannt wurden. In einer realen Umgebung müsste also die ganze Capture Datei manuell durchgearbeitet werden, um die Erkennung zu überprüfen und Massnahmen zu ergreifen.

Deshalb brauchte der Prototyp einen Weg, erkannte Anomalien wieder zu Paketdetails umzuwandeln. Da die NFStream Library direkt Flows zurückgibt, und nie direkte Paketdetails ausgab, wusste ich zuerst nicht ob es überhaupt möglich ist. Ich fand dann aber heraus, dass NFStream eine Klasse namens NFPlugin hatte, mit der man einfache Erweiterungen hinzufügen konnte. So konnte ich eine kleine Erweiterung schreiben, die bei jedem Flow ein Attribut hinzufügt, dass pro Paket die IP-Adressen und Portnummern speichert. So kann man später mit diesen Informationen auf aufgezeichnete Pakete zugreifen.

Da ich aber später die Flows noch in Cluster gruppiert hatte, musste ich meinen Prototypen noch erweitern, dass von einem Cluster auf jeden Flow zurückgegriffen werden konnte. Am Ende speichert mein Prototyp die Paketdaten in einer Excel-Datei, zur Überprüfung und Analyse.

2.5 Testen mit echten C2 Frameworks

Für das Testen mit echten Daten brauchte ich ein C2 Framework, dass ich in einer Testumgebung aufbauen konnte. Zuerst wollte ich Cobalt Strike verwenden, aber leider ist Cobalt Strike nicht Gratis. Deswegen entschied ich mich für das C2 Framework Sliver, welches Gratis und Open Source ist.

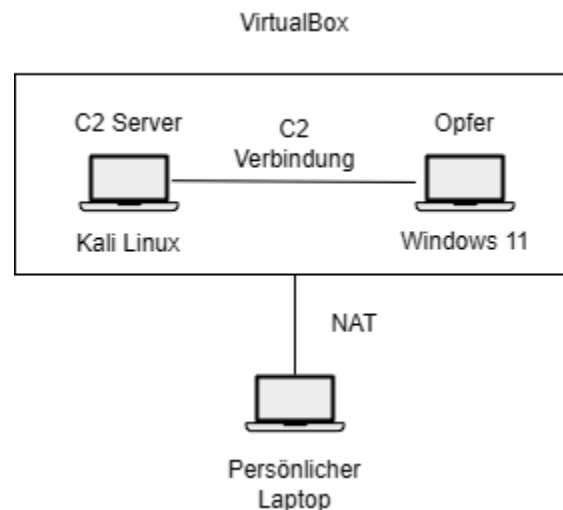


Abbildung 20: Netzwerkplan Testumgebung

Für meine Testumgebung nutzte ich eine Kali Linux VM als C2 Server und eine Windows 11 VM als Opfer. Auf dem Windows 11 System zeichnete ich den Datenverkehr mit Wireshark auf.

2.5.1 Sliver Setup

Sliver ist sehr einfach aufzusetzen, der Installationsbefehl ist «`sudo apt install sliver-server`». Danach kann Sliver mit dem Befehl «`sliver-server`» gestartet werden.

Ein Beacon für Windows kann mit dem Befehl «`generate beacon -b <attacker ip> --os windows`». Es gibt noch viele weitere Optionen, aber für einen einfachen Beacon reicht dies. In der Ausgabe des Befehls sieht man, an welchem Ort die generierte Datei abgelegt wurde. Dies ist normalerweise das Homeverzeichnis des Benutzers.

Mit dem Befehl «`http`» kann man einen http Listener starten. Mit dem Befehl «`jobs`» kann man nochmals überprüfen, ob der Listener aktiv ist.

```
[server] sliver > http

[*] Starting HTTP :80 listener ...
[!] rpc error: code = AlreadyExists desc = port 80 is in use

[server] sliver > jobs

  ID   Name   Protocol   Port   Domains
  ==   ==   ==
  1    http    tcp        80
[server] sliver > 
```

Abbildung 21: Sliver HTTP Listener

Nachdem der erstellte Beacon auf dem Windows System ausgeführt wurde, sah ich, dass der C2 Server erfolgreich die Verbindungen entgegennahm. Auf dem Windows Client habe ich dann angefangen, den Datenverkehr mit Wireshark aufzuzeichnen.

Zuerst war ich ungeduldig und habe nach 5'000 Paketen aufgehört aufzuzeichnen, aber dies gab sehr inkonsistente Resultate bei meinem Prototyp, weshalb ich es nochmals machen musste. Unten sieht man die Auswertung meines Prototyps mit dem Sliver C2 Verkehr und ungefähr 90'000 Paketen.

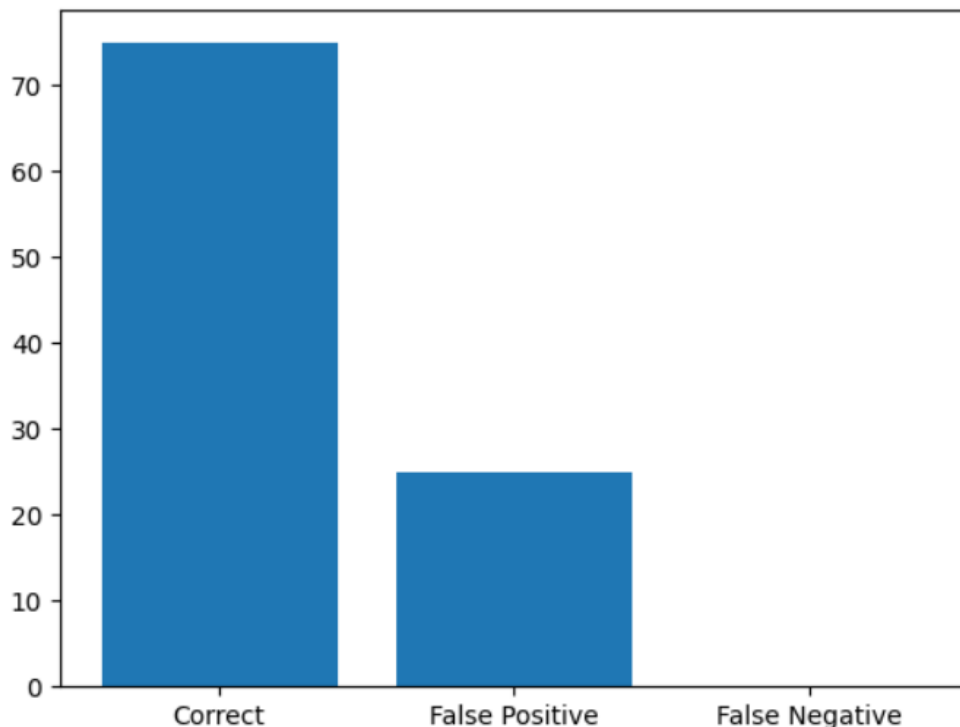


Abbildung 22: Sliver C2 Verkehr

Der Prototyp funktionierte schon sehr gut, und zusammen mit einer angepassten Einstellung konnte der Prototyp sogar 100 von 100 Versuche richtig erkennen.

Halluzinationen

Ein Problem das ich bemerkte bei meinem Prototyp war, dass wenn ich eine Capture Datei nutzte die keinen C2 Verkehr enthielt, der Prototyp trotzdem Pakete als Anomalien erkannte.

Glossar

Capture File/Datei

C2

Command & Control

Feature

Library

Tagesreflexionen

Tag 1, 03.06

Am ersten Tag der Projektarbeit habe ich mein Github Repository und Project erstellt. Dann musste ich meine Meilensteine aufschreiben und ich habe mir noch ein optionales Ziel überlegt. Danach habe ich angefangen mit der Planung und danach meine Issues hinzugefügt.

Tag 2, 04.06

Am zweiten Tag habe ich einige Issues zur Informationsbeschaffung abgearbeitet. Ich habe angeschaut was für Techniken und Taktiken von Angreifern verwendet werden, um nicht erkannt zu werden. Ich habe auch nach öffentlichen Tools gesucht, die etwas ähnliches bewerkstelligen, um ein wenig Inspiration zu kriegen. Als Letztes habe ich nach Informationen über verschiedene ML-Algorithmen gesucht, um zu entscheiden welcher der beste für mein Projekt ist.

Tag 3, 05.06

Am dritten Tag habe ich die letzten Issues zur Informationsbeschaffung bearbeitet. Ich verwendete den Grossteil des Tages darauf, herauszufinden welche Features nötig oder unnötig sind. Ich hatte einige Probleme mit meinen Scripts, da ich auch nicht so erfahren darin bin, mit grossen Datenmengen und spezifischen Librarys zu arbeiten.

Tag 4, 10.06

Am vierten Tag habe ich zum ersten Mal den Isolation Forest Algorithmus benutzt. Zuerst habe ich mit Testdaten ausprobiert, wie der Algorithmus funktioniert und welche Parameter es gibt. Danach habe ich meine schon erstellten

Tag 5, 11.06

Verbesserung mit kombination

Tag 6, 12.06

verbesserung mit cluster

Tag 7, 17.06

setup sliver, testen sliver

Tag 8, 18.06

neuer aufbau testen, halluzinationen bekämpfen

Tag 9, 19.06

Tag 10, 24.06

Tag 11, 25.06

Tag 12, 26.06